

1. Right-aligned increasing triangle

Input the number of rows, n, and print a right-aligned increasing triangle using asterisks.

For n = 4:

```
  *
 **
***
****
```

Algorithm:

1. Start
2. Declare variables n, i, j, k.
3. Read input value for number of rows n.
4. Run outer loop i from 1 to n for each row:
 - Run inner loop j from 1 to (n - i) to print spaces.
 - Run inner loop k from 1 to i to print stars '*'.
 - Print newline character.
5. Stop

Test Case Table:

Test Case	Input (n)	Expected Output	Actual Result
1	4	4 rows right aligned triangle of *	As Expected
2	1	*	As Expected
3	0	No output printed	As Expected

2. Right-aligned increasing number triangle

Input the number of rows, n, and print a right-aligned triangle of natural numbers. In each row, print the natural numbers from 1 up to the corresponding row number.

For n = 4:

```
  1
 1 2
1 2 3
1 2 3 4
```

Algorithm:

1. Start

2. Declare variables n, i, j, k.
3. Read value of n.
4. Loop i from 1 to n:
 - Loop j from 1 to (n - i): print space.
 - Loop k from 1 to i: print value of k.
 - Print newline.
5. Stop

Test Case Table:

Test Case	Input (n)	Expected Output	Actual Result
1	4	4 rows number pattern starting 1 to row no.	As Expected
2	3	3 rows number pattern	As Expected

3. Right-aligned decreasing triangle

Input the number of rows, n, and print a right-aligned decreasing triangle using asterisks.

For n = 4:

```
* * * *
 * * *
  * *
   *
```

Algorithm:

1. Start
2. Declare variables n, i, j, k.
3. Read input n.
4. Loop i from n down to 1:
 - Loop j from 1 to (n - i): print space.
 - Loop k from 1 to i: print '*'.
 - Print newline.
5. Stop

Test Case Table:

Test Case	Input (n)	Expected Output	Actual Result
1	4	4 rows decreasing right aligned * pattern	As Expected
2	2	2 rows decreasing pattern	As Expected

C Program Code:

```
#include <stdio.h>

void pattern1();
void pattern2();
void pattern3();

int main() {
    int choice;

    while(1) {
        printf("
--- MENU ---
```

```

");
    printf("1. Right-aligned increasing triangle
");
    printf("2. Right-aligned increasing number triangle
");
    printf("3. Right-aligned decreasing triangle
");
    printf("4. Exit
");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch(choice) {
        case 1:
            pattern1();
            break;
        case 2:
            pattern2();
            break;
        case 3:
            pattern3();
            break;
        case 4:
            return 0;
        default:
            printf("Invalid choice!
");
    }
}
return 0;
}

void pattern1() {
    int n, i, j, k;
    printf("Enter n: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for(k = 1; k <= i; k++) {
            printf("*");
        }
        printf("
");
    }
}

```

```

    }
}

void pattern2() {
    int n, i, j, k;
    printf("Enter n: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for(k = 1; k <= i; k++) {
            printf("%d ", k);
        }
        printf("
");
    }
}

void pattern3() {
    int n, i, j, k;
    printf("Enter n: ");
    scanf("%d", &n);

    for(i = n; i >= 1; i--) {
        for(j = 1; j <= n - i; j++) {
            printf(" ");
        }
        for(k = 1; k <= i; k++) {
            printf("*");
        }
        printf("
");
    }
}

```